Note 2: Version Conflicts, Dependency Management & Resolution

Version Conflicts

How Conflicts Arise

Because Maven resolves transitive dependencies automatically, it is common for two different dependencies to require different versions of the same library. For example:

- Library A requires jackson-databind:2.13.0
- Library B requires jackson-databind:2.15.0

Both cannot be on the classpath at the same time. Maven must choose one version.

Maven's Conflict Resolution Strategy — Nearest Wins

Maven uses the nearest definition wins rule:

- The dependency that is closest to your project in the dependency tree is chosen
- If two dependencies are at the same depth, the one declared first in the POM wins

This can lead to subtle runtime bugs if the wrong version is chosen — a method that exists in version 2.15.0 may not exist in 2.13.0, causing a NoSuchMethodError at runtime.

How to Detect Conflicts

mvn dependency:tree

mvn dependency:analyze

dependency:analyze reports dependencies that are declared but unused, and dependencies that are used but not explicitly declared.

Resolving Version Conflicts

Method 1: Explicit Declaration in your POM The simplest fix is to explicitly declare the version you want in your own POM's <dependencies> section. Since your project is at depth 1 (the nearest), your declared version always wins.

```
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.15.0</version>
```

```
  </dependency>
</dependencies>
```

Method 2: Using <dependencyManagement> The <dependencyManagement> section is the preferred approach in multi-module projects. It allows you to declare version numbers centrally without actually adding those dependencies to the project. Child modules then inherit the version without specifying it.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.0</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

The key difference between <dependencies> and <dependencyManagement>:

- <dependencies> — actually adds the dependency to the project
- <dependencyManagement> — only sets the version to use; the dependency must still be declared in <dependencies> (without a version) to be included

Method 3: Excluding a Transitive Dependency If a specific transitive dependency is causing a conflict, you can exclude it from the library that brings it in:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>library-a</artifactId>
  <version>1.0.0</version>
  <exclusions>
    <exclusion>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
```

```
    </exclusion>
  </exclusions>
</dependency>
```

Then declare the version you actually want separately in your <dependencies> section.

Method 4: Using a BOM (Bill of Materials) A BOM is a special POM that declares a curated, tested set of dependency versions that are all known to work together. Spring Boot provides a BOM. You import it using <scope>import</scope> in <dependencyManagement>:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>3.1.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

After this, you can declare any Spring Boot dependency without a version and the BOM-specified version will be used automatically.

Note 3: Maven Plugins, Execution & Maven Wrapper

Maven Plugins

What are Maven Plugins?

Almost everything Maven does is implemented through plugins. The build lifecycle phases (compile, test, package, etc.) are all executed by plugins bound to those phases. Maven itself is just a framework — the actual work is done by plugins.

Plugins consist of goals — a goal is a specific task a plugin can perform. Goals are bound to lifecycle phases so they execute automatically, or they can be called directly.

Syntax to call a plugin goal directly:

mvn plugin-name:goal-name

mvn dependency:tree

mvn compiler:compile

Maven Compiler Plugin

The Compiler Plugin is responsible for compiling Java source files. It is bound to the compile and test-compile phases automatically.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
        <encoding>UTF-8</encoding>
      </configuration>
    </plugin>
  </plugins>
</build>
```

This tells Maven to compile source code using Java 17 syntax. If your pom.xml already sets maven.compiler.source and maven.compiler.target in <properties>, this plugin configuration is applied automatically without needing to declare the plugin explicitly.

Surefire Plugin (Unit Testing)

The Surefire Plugin runs unit tests during the test phase. It discovers test classes automatically based on naming conventions and generates XML and plain text reports in target/surefire-reports/.

Default naming conventions for test classes Surefire picks up:

- **/*Test.java
- **/Test*.java
- **/*Tests.java
- **/*TestCase.java

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
  <configuration>
    <includes>
      <include>**/*Test.java</include>
    </includes>
    <skipTests>false</skipTests>
  </configuration>
</plugin>
```

To run only a specific test class:

```
mvn test -Dtest=UserServiceTest
```

To run only a specific test method:

```
mvn test -Dtest=UserServiceTest#shouldReturnUserById
```

Shade Plugin (Uber JAR)

By default, mvn package creates a JAR that contains only your compiled classes — it does not include your dependencies. This JAR cannot be run standalone because the dependencies are missing. The Shade Plugin creates an uber JAR (also called a fat JAR) — a single JAR that contains your classes plus all dependency classes bundled together.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
```

```
  <version>3.5.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>shade</goal>
      </goals>
      <configuration>
        <transformers>
          <transformer implementation=
          "org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>com.example.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>
```

The <mainClass> specifies which class contains the main() method so the JAR can be executed with:

java -jar target/my-app-1.0.0.jar

The Shade plugin binds to the package phase, so running mvn package automatically produces the uber JAR.

Maven Wrapper (mvnw)

What is the Maven Wrapper?

The Maven Wrapper is a script (mvnw on Linux/Mac, mvnw.cmd on Windows) that is checked into your project's source control alongside your code. It allows anyone to build the

project using a specific, project-defined version of Maven — without needing Maven to be installed on their machine.

When you run ./mvnw install, the wrapper:

1. Checks if the required Maven version is already downloaded in ~/.m2/wrapper/
2. If not, downloads that exact version from the internet automatically
3. Uses that downloaded version to run the build

Why the Wrapper Matters

- Eliminates the "works on my machine" problem caused by different Maven versions
- CI/CD pipelines can use ./mvnw without needing Maven pre-installed on the build agent
- The exact Maven version is pinned in .mvn/wrapper/maven-wrapper.properties, making builds fully reproducible

Files Added by the Maven Wrapper

my-project/

├── mvnw ← Shell script for Linux/Mac

├── mvnw.cmd ← Batch script for Windows

└── .mvn/

  └── wrapper/

    └── maven-wrapper.properties

maven-wrapper.properties

distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.4/apache-maven-3.9.4-bin.zip

wrapperUrl=https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper/3.2.0/maven-wrapper-3.2.0.jar

This file specifies the exact Maven version to download and use.

Generating the Maven Wrapper

If you have Maven installed locally, you can add the wrapper to any project:

mvn wrapper:wrapper

Or specify a version:

mvn wrapper:wrapper -Dmaven=3.9.4

Using the Maven Wrapper

On Linux/Mac:

./mvnw clean install

./mvnw test

./mvnw package

On Windows:

mvnw.cmd clean install

mvnw.cmd package

All standard Maven commands work exactly the same — just replace mvn with ./mvnw.

Plugin Execution Summary

Plugin	Phase	Purpose
maven-compiler-plugin	compile	Compiles Java source files
maven-surefire-plugin	test	Runs unit tests, generates reports
maven-shade-plugin	package	Creates uber JAR with all dependencies
maven-wrapper-plugin	—	Generates mvnw scripts for wrapper setup

Note 2: Version Conflicts, Dependency Management & Resolution

Version Conflicts

How Conflicts Arise

Because Maven resolves transitive dependencies automatically, it is common for two different dependencies to require different versions of the same library. For example:

- Library A requires jackson-databind:2.13.0
- Library B requires jackson-databind:2.15.0

Both cannot be on the classpath at the same time. Maven must choose one version.

Maven's Conflict Resolution Strategy — Nearest Wins

Maven uses the nearest definition wins rule:

- The dependency that is closest to your project in the dependency tree is chosen

- If two dependencies are at the same depth, the one declared first in the POM wins

This can lead to subtle runtime bugs if the wrong version is chosen — a method that exists in version 2.15.0 may not exist in 2.13.0, causing a NoSuchMethodError at runtime.

How to Detect Conflicts

mvn dependency:tree

mvn dependency:analyze

dependency:analyze reports dependencies that are declared but unused, and dependencies that are used but not explicitly declared.

_____

Resolving Version Conflicts

Method 1: Explicit Declaration in your POM The simplest fix is to explicitly declare the version you want in your own POM's <dependencies> section. Since your project is at depth 1 (the nearest), your declared version always wins.

```
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.15.0</version>
  </dependency>
</dependencies>
```

Method 2: Using <dependencyManagement> The <dependencyManagement> section is the preferred approach in multi-module projects. It allows you to declare version numbers centrally without actually adding those dependencies to the project. Child modules then inherit the version without specifying it.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.0</version>
```

```
    </dependency>
  </dependencies>
</dependencyManagement>
```

The key difference between <dependencies> and <dependencyManagement>:

- <dependencies> — actually adds the dependency to the project
- <dependencyManagement> — only sets the version to use; the dependency must still be declared in <dependencies> (without a version) to be included

Method 3: Excluding a Transitive Dependency If a specific transitive dependency is causing a conflict, you can exclude it from the library that brings it in:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>library-a</artifactId>
  <version>1.0.0</version>
  <exclusions>
    <exclusion>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Then declare the version you actually want separately in your <dependencies> section.

Method 4: Using a BOM (Bill of Materials) A BOM is a special POM that declares a curated, tested set of dependency versions that are all known to work together. Spring Boot provides a BOM. You import it using <scope>import</scope> in <dependencyManagement>:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
```

```
            <version>3.1.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

After this, you can declare any Spring Boot dependency without a version and the BOM-specified version will be used automatically.
